

Week 1: Running the Programs in VS Code

Dr M AOUDE

Guide to the week 1 programs

About this guide. It explains how to run the week 1 programs in **Visual Studio Code**, and gives a short description of every program in the `code/week01` folder: the programs from the slides, the programs from the lecture notes, and the Flutter examples. Each program file also starts with a comment that says what it teaches and what output to expect: run it, then compare your result with that comment.

1 What you need

- **Flutter** installed. It includes the Dart SDK, so the `dart` command works too. Check it in a terminal: `dart --version`. Installation steps are in the install guides `001_install_vscode` (VS Code edition) and `001_install` (Android Studio edition).
- **Visual Studio Code** with the **Flutter** extension (`Dart-Code.flutter`). It installs the Dart extension too.
- **Git**, only if you want to download the code with `git clone`.

2 Get the code and open it in VS Code

Download the course repository, or copy it with Git:

```
git clone https://github.com/maoude/mobile_pro_2026.git
```

In VS Code choose **File** → **Open Folder...** and open the folder `code/week01` (or the whole repository). The Explorer on the left now shows three folders:

Folder	Content
<code>slides</code>	the programs shown in the week 1 slides (plain Dart)
<code>lect_01_01</code>	the programs from the lecture notes (plain Dart)
<code>flutter_examples</code>	the Flutter widgets from the lecture notes (a small Flutter project)

3 Run a Dart program

The programs in `slides` and `lect_01_01` are plain Dart programs. There are two ways to run one in VS Code.

Way 1: the Run button

1. Open the file from the Explorer, for example `slides/01_hello_world.dart`.
2. Click **Run** in the small **Run | Debug** line that appears just above `void main()`. You can also press `F5`, or `Ctrl+F5` to run without the debugger.
3. The output appears in the **Debug Console** panel at the bottom.

Way 2: the terminal

1. Open a terminal with **Terminal** → **New Terminal** (or Ctrl+').
2. Go to the folder of the program and run it with `dart run`:

```
cd code/week01/slides
dart run 01_hello_world.dart
```

Use the terminal for the programs that read from the keyboard (next section). Slides 12 to 14 also show how to compile a program to an executable file:

```
dart compile exe -o test.exe test.dart
test
```

4 Programs that ask you to type

Two programs read what you type:

- 06_console_input_names.dart
- 07_console_input_sum.dart

The Debug Console cannot receive keyboard input, so run them in the **terminal** (Way 2), type your answer after each question, and press Enter:

```
cd code/week01/slides
dart run 07_console_input_sum.dart
```

(If you prefer the Run button, add "console": "terminal" to the launch configuration in .vscode/launch.json: the program then runs in the terminal.)

5 Programs from the slides

Folder: code/week01/slides. Run each one with `dart run <file>`.

1. Hello world

01_hello_world.dart

The smallest Dart program. Execution starts in the `main` function, and `print` writes a line of text on the console.

```
hello world
```

2. Adding two numbers with var

02_add_numbers_var.dart

Adds two numbers stored in `var` variables. Dart infers that they are integers, and the program prints the type it chose. It also shows string interpolation: `$sum` puts the value of a variable inside a string.

```
Result: 30
Type of x: int
Type of sum: int
```

3. Adding two numbers with explicit types

03_add_numbers_typed.dart

The same program with the types written out (`int`). It also shows that the division operator `/` always gives a `double`, while `~/` gives only the whole-number part.

```
Result: 30
Average (/): 15.0
Integer division (~/): 15
```

4. Dart is strongly typed**04_type_error.dart**

Once Dart infers that a variable is an `int`, you cannot store text in it. The program runs as it is. Remove the two slashes in front of the line marked “TRY IT” to see the compile-time error from the slide, then put them back. It also shows `dynamic`, the exception where the type may change.

```
Value: 10
Value: 20
dynamic can change type: some text
```

5. Even numbers between m and n**05_even_numbers.dart**

Prints the even numbers from 10 to 20 in two ways: with a `for` loop that tests every number with `i % 2 == 0`, and with a faster loop that starts at the first even number and jumps by 2. Both print 10, 12, 14, 16, 18, 20.

6. Console input: names**06_console_input_names.dart**

Asks for a first and a last name with `stdin.readLineSync()` and greets you. The variables are `String?` because the input can be null. Run it in the terminal.

```
Enter first name: Ali
Enter last name: Khan
Welcome Ali Khan
```

7. Console input: adding two integers**07_console_input_sum.dart**

Reads two numbers as text and converts them with `int.parse`. The `!` operator tells Dart the input is not null. Typing something that is not a whole number stops the program with an error. Run it in the terminal.

```
Enter x: 10
Enter y: 20
Sum is 30
```

6 Programs from the lecture notes

Folder: `code/week01/lect_01_01`. These are plain Dart programs too; run each one with `dart run <file>`.

1. Variables and collections**01_variables_and_collections.dart**

Compares `var`, `final` (assigned once, at run time) and `const` (a compile-time constant). It builds a `List`, a `Map` and a `Set`, and shows that a `final` list can still grow while a `const` list cannot be changed.

2. Switch and pattern matching**02_switch_pattern_matching.dart**

Dart 3 switch expressions: several patterns can share one result (`||`), `_` matches everything else, and relational patterns such as `>= 90` turn a score into a grade. Example lines: `Flutter -> Cross-platform`, `Score 95 -> A`.

3. A user model**03_user_model.dart**

An immutable `User` class with `final` fields, named and `required` parameters, and a default value for the creation date. It rejects an invalid email, and converts a user to JSON text and back (`toJson` and `fromJson`).

4. Futures and `async/await`**06_futures_async_await.dart**

Simulated network calls, each taking 300 ms. It shows `await` for a single call, `Future.wait` to run three calls in parallel (about 300 ms instead of 900 ms), a type-safe version with a record of futures, and error handling with `try`, `catch` and `finally`.

5. Streams and cleanup**08_streams_and_cleanup.dart**

A `StreamController` sends events to a listener. The program then cancels the subscription and closes the controller: the cleanup that every stream needs.

6. Null safety**09_null_safety_basics.dart**

`String` can never be null while `String?` can. It uses `??` for a fallback value and `?.` for a safe member access, and shows what happens when a `late` variable is read before it is set.

7. Extension methods**12_extension_methods.dart**

Adds new members to existing types: `isValidEmail` and `capitalizeFirst()` for `String`, and `20.0.px` for `double`.

8. Records and sealed classes**13_records_and_sealed_classes.dart**

Records group values without a class, as named or positional values. A sealed `LoadState` class (loading, success, failure) must be handled completely by a switch, so no case is forgotten.

9. Quick start**19_quick_start.txt**

Not a Dart program: the three shell commands that create and run a Flutter project (`flutter create`, `cd`, `flutter run`), with an explanation of each and of what you should see.

7 The Flutter examples

Folder: `code/week01/flutter_examples`. This is a small Flutter project; each file in `lib` is a complete app with its own `main`. Prepare it once, in the terminal (**Terminal** → **New Terminal**) opened in that folder:

```
flutter pub get
flutter create .
```

`flutter pub get` downloads the packages, and `flutter create .` adds the platform folders (it does not change the existing files).

Run an example

1. Choose the device at the bottom right of the VS Code window: **Chrome**, **Windows (desktop)**, an emulator, or your phone.
2. Open an example, for example `lib/15_counter_button.dart`, and click **Run** above its `main` (or press F5).
3. While the app runs, save the file to hot reload it. `Ctrl+Shift+F5` does a hot restart.

The same from the terminal: `flutter run -t lib/15_counter_button.dart`. To run the automated checks that verify every example, type `flutter test`.

What each example shows

File	What you see
04_profile_card.dart	A card built from ready-made widgets: avatar with a letter, name and email. Tap it for a message.
05_dependency_injection.dart	A spinner, then three user cards. The data comes through Provider .
07_future_builder.dart	The same list, loaded with a FutureBuilder : spinner, then data.
08_stream_cleanup.dart	A counter that goes up each time you tap the + button (a stream at work).
10_late_bad_vs_good.dart	The text “ready”. Swap in BadExample to see the late crash.
13_load_state_ui.dart	A spinner, the number 42, and Error: timeout: one widget per state.
14_widget_tree.dart	The word “Hello” in the centre. It is also the default app of lib/main.dart .
15_counter_button.dart	A button that shows Count: 0 , then 1, 2, 3 as you tap it.
16_lifecycle.dart	Ticks counting up every second, and the app’s lifecycle state.
17_responsive_grid.dart	A grid of 12 cards: 2 columns in a narrow window, 3 in a wide one.
18_theme.dart	An app bar and a button coloured from one blue seed colour.

8 Quick troubleshooting

Problem	What to do
dart is not recognised	Close all terminals and restart VS Code so it sees the new PATH; check with flutter doctor . See the install guide.
No Run Debug line above main	Install the Flutter (Dart) extension, and open the folder that contains the file (Section 2).
A program waits and nothing happens	It is waiting for input: run it in the terminal, not the Debug Console (Section 4).
The Flutter example has no device to run on	Choose Chrome at the bottom right of VS Code, or start an emulator.

This document is maintained by Dr. Mohamad AOUE

Last updated: 2026-09-20